

Lesson 2. Programming with Numbers and Strings

Lập trình với số và chuỗi

Lesson focus

- Variables, constants, numeric types, and assignment.
- Arithmetic expressions, strings, and input/output.
- Problem-solving workflow: hand calculation → algorithm → Python.

Learning objectives

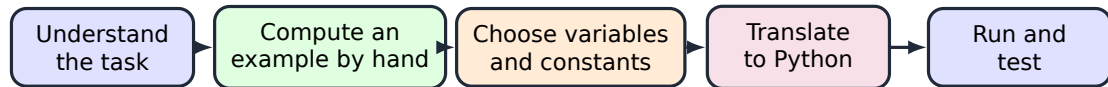
After this lesson, students should be able to:

- use **variables**, **constants**, int, and float;
- write expressions with +, -, *, /, //, %, and **;
- work with strings: concatenation, repetition, indexing, and methods;
- read input, convert data types, and format output;
- solve problems using the **Input** → **Process** → **Output** model.

Lesson structure

- 1 Variables
- 2 Arithmetic
- 3 Problem solving: first do it by hand
- 4 Strings
- 5 Input and output
- 6 Optional tool: SymPy
- 7 Summary and exercises

Programming workflow in Chapter 2



Key idea

Do not start by typing random Python statements. First understand the computation.

What is a variable?

Definition

A **variable** is a named storage location in a program.

- the name identifies the storage location;
- the value is the current content stored there;
- a variable can be used later in computations.

Variable name

Stored value



Think of a variable as a labeled box: the label is the name, and the content is the value.

Assignment statements

General form

```
variableName = value
```

```
cansPerPack = 6  
print(cansPerPack)
```

Meaning

The first assignment creates the variable and initializes it with a value.

Important

Python executes statements from top to bottom.

Assignment is not mathematical equality

```
cansPerPack = 8  
cansPerPack = cansPerPack + 2  
print(cansPerPack)
```

Execution steps

- 1 Read current value: 8.
- 2 Compute $8 + 2 = 10$.
- 3 Store 10 back into cansPerPack.

Key idea

Read = as **assign**, not as mathematical equality.

Numeric data types

Every value has a **data type**. The type determines how the value is represented and which operations are valid.

Python type	Meaning	Examples
int	integer / whole number	6, 0, -25
float	floating-point number	0.355, 1.0, -3.5

- 6 is an int; 6.0 is a float.
- The type belongs to the value, not permanently to the variable name.

Variable names

Rules

- start with a letter or underscore;
- then use letters, digits, or underscores;
- no spaces or symbols such as ?, %, /;
- names are case sensitive;
- reserved words cannot be used.

Name	Valid?
canVolume1	yes
x	yes, but vague
CanVolume	yes, but different
6pack	no
can volume	no
class	no

Constants and comments

Constants

A constant is a value that should remain unchanged during the program.

```
BOTTLE_VOLUME = 2.0  
MAX_SIZE = 100
```

Comments

Comments explain code for human readers.

```
CAN_VOLUME = 0.355 # liters per can
```

Python ignores text after # on that line.

Example: computing soda volume

```
##  
# This program computes the volume of a six-pack  
# of soda cans and a two-liter bottle.  
#  
CAN_VOLUME = 0.355  
BOTTLE_VOLUME = 2.0  
  
cansPerPack = 6  
totalVolume = cansPerPack * CAN_VOLUME  
print("A six-pack contains", totalVolume, "liters.")  
  
totalVolume = totalVolume + BOTTLE_VOLUME  
print("Together they contain", totalVolume, "liters.")
```

Common Error 2.1: using undefined variables

Incorrect

```
canVolume = 12 * literPerOunce  
literPerOunce = 0.0296
```

Correct

```
literPerOunce = 0.0296  
canVolume = 12 * literPerOunce
```

Error

```
NameError: name 'literPerOunce' is  
not defined
```

- define before use;
- check spelling and capitalization.

Self Check 2.1: variables and assignment

Concept questions

- 1 What happens the first time `count = 10` executes?
- 2 What is the difference between 6 and 6.0?
- 3 Why should constants use descriptive uppercase names?

Quick check

- 1 If `x = 7` and then `x = x + 3`, what is `x`?
- 2 Why is `result = price * quantity` wrong if `quantity` has not been assigned yet?

Short programming exercise: variables and constants

Task

Write a short program that computes the total volume of soda.

- **Input/known values:** number of cans, liters per can, bottle volume.
- **Process:** compute can volume and total volume.
- **Output:** print the total liters.

```
CAN_VOLUME = 0.355  
BOTTLE_VOLUME = 2.0  
cansPerPack = 6  
# complete the computation
```

Basic arithmetic operations

Operation	Python operator	Example
Addition	+	<code>a + b</code>
Subtraction	-	<code>a - b</code>
Multiplication	*	<code>a * b</code>
Division	/	<code>a / b</code>

Expression

An expression combines literals, variables, operators, parentheses, and function calls to compute a value.

Operator precedence

Order

- 1 Parentheses
- 2 Exponentiation
- 3 Multiplication and division
- 4 Addition and subtraction

$a + b / 2$	# $b / 2$ first
$(a + b) / 2$	# average of a and b

Use parentheses

Parentheses make the intended calculation explicit.

Powers

Python uses `**` for exponentiation.

```
2 ** 3          # 8
10 * 2 ** 3     # 80
10 ** 2 ** 3
```

Important

Exponentiation has higher precedence than multiplication.

Math to Python

$$b \left(1 + \frac{r}{100}\right)^n$$

```
b * (1 + r / 100) ** n
```

Floor division and remainder

Normal division

$7 / 4$ # 1.75

Remainder

$7 \% 4$ # 3

Floor division

$7 // 4$ # 1

Use case

These operators are useful for money, time conversion, digits, and packing problems.

Useful patterns with // and %

For a positive integer n :

Pattern	Meaning
$n \% 10$	last digit
$n // 10$	all digits except the last digit
$n \% 100$	last two digits
$n \% 2$	0 if even, 1 if odd

Example

If $n = 1729$, then $n // 100 = 17$ and $n \% 100 = 29$.

Calling functions

A function can return a value that can be stored, printed, or used in an expression.

```
distance = abs(-173)
print(round(7.627, 2))
print(min(7.25, 10.95, 5.95, 6.05))
```

Function	Purpose
abs(x)	absolute value
round(x, n)	round to n decimal places
max(...)	largest value
min(...)	smallest value

The math module

Import one function

```
from math import sqrt  
  
y = sqrt(25)  
print(y)
```

Common functions

- `sqrt(x)`: square root
- `sin(x)`, `cos(x)`, `tan(x)`
- `log(x)`, `exp(x)`
- `radians(x)`, `degrees(x)`

Common errors and expression-writing tips

Roundoff error

```
price = 4.35  
quantity = 100  
print(price * quantity)
```

The result may be close to, but not exactly, 435.

Unbalanced parentheses

```
((a + b) * t / 2 * (1 - t)
```

Check that every opening parenthesis has a matching closing parenthesis.

Tip

Use spaces and parentheses to make expressions readable.

Special Topic 2.1: importing modules

Selected names

```
from math import sqrt,  
    sin, cos
```

Use functions directly.

All names

```
from math import *
```

Convenient but less explicit.

Module name

```
import math  
math.sqrt(x)
```

Shows where the function comes from.

Special Topic 2.2: augmented assignment

Short form

```
total += cans  
count += 1  
total *= 2
```

Equivalent form

```
total = total + cans  
count = count + 1  
total = total * 2
```

Meaning

The variable must already have a value before augmented assignment is used.

Special Topic 2.3: line joining

Long expressions can be split across lines inside parentheses.

Readable

```
x1 = ((-b + sqrt(b ** 2 - 4 * a * c))  
      / (2 * a))
```

Why useful?

- avoids very long lines;
- keeps the expression structure visible;
- reduces parentheses mistakes.

Self Check 2.2: arithmetic and expressions

Concept questions

- 1 What is the value of $10 + 6 / 2$?
- 2 What are the results of $17 // 5$ and $17 \% 5$?
- 3 Why is $a + b / 2$ different from $(a + b) / 2$?

Quick check

Write Python expressions for: average of a and b ; $a^2 + b^2$; and $\sqrt{x^2 + y^2}$.

Short programming exercise: seconds conversion

Task

Read a number of seconds and convert it to minutes and remaining seconds.

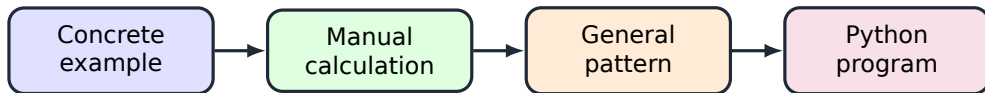
- **Input:** total seconds.
- **Process:** use // and %.
- **Output:** full minutes and remaining seconds.

```
seconds = int(input("Seconds: "))
minutes = seconds // 60
remaining = seconds % 60
print(minutes, remaining)
```

First do it by hand

Principle

If you cannot compute a solution yourself, it is unlikely that you can write a reliable program that automates it.



Example: alternating tiles

Problem summary

- Place black and white tiles along a wall.
- First tile: black.
- Colors alternate.
- Last tile should also be black.

Example

- total width = 100 inches;
- tile width = 5 inches;
- pattern: B | WB | WB | ... | WB.

Result

Number of pairs: $95 // 10 = 9$. Total tiles: $1 + 2 * 9 = 19$.

Worked Example 2.1: computing travel time

Problem

A robot moves to an item. It moves faster on a road and slower across rocky terrain.

Input / Output

Input: distances, speeds, first segment length.

Output: total travel time.

Computation

$$\text{time}_1 = \frac{\text{length}_1}{\text{speed}_1}$$

$$\text{length}_2 = \sqrt{(x - \text{length}_1)^2 + y^2}$$

$$\text{total} = \text{time}_1 + \text{length}_2 / \text{speed}_2$$

Self Check 2.3

- 1 Why should we work through a concrete example before coding?
- 2 In the tile example, why is floor division useful?
- 3 Which order is better: code first, or example $\rightarrow$ algorithm $\rightarrow$ code?

What is a string?

A **string** is a sequence of characters.

```
"Hello"  
"Python"  
"123 Main Street"
```

Characters can include

letters, digits, punctuation, spaces, Unicode symbols, and emoji.

```
greeting = "Hello"  
print(greeting)  
print(len("World!"))
```

Concatenation, repetition, and conversion

Concatenation

```
first = "Harry"  
last = "Morgan"  
name = first + " " + last
```

Repetition

```
dashes = "-" * 50  
message = "Echo. "  
print(message * 5)
```

Conversion

```
id = 1729  
name = "Agent " + str(id)  
price = float("17.29")
```

Strings and indices

Python starts string indices at 0.

String	H	a	r	r	y
Index	0	1	2	3	4

```
name = "Harry"  
first = name[0]  
last = name[len(name) - 1]
```

Important

An index outside the valid range causes an exception while the program runs.

String methods

A method is an operation associated with an object.

```
name = "John Smith"  
uppercaseName = name.upper()  
lowercaseName = name.lower()  
name2 = name.replace("John", "Jane")
```

Key idea

String methods return new strings. They do not modify the original string object directly.

Unicode and escape sequences

Unicode

Python 3 strings can represent characters from many writing systems.

```
text = "£100"  
print(text[0])
```

Escape sequences

```
"You're \"Welcome\""  
"C:\\Temp\\Secret.txt"  
print("*\\n**\\n***")
```

Self Check 2.4: strings

Concept questions

- 1 What is the value of `len("Hello")`?
- 2 What is the result of `"Py" + "thon"`?
- 3 What is the result of `"-" * 5`?

Quick check

Given `s = "Python"`, identify `s[0]` and `s[len(s)-1]`.

Short programming exercise: string processing

Task

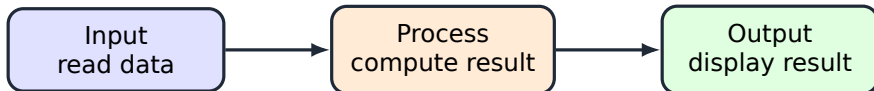
Read a word and print its first character, last character, and length.

```
word = input("Word: ")
print("First:", word[0])
print("Last:", word[len(word) - 1])
print("Length:", len(word))
```

Assumption

For this exercise, assume the user enters a non-empty word.

The Input → Process → Output model



Example

A program may read price and quantity, compute total cost, and display the invoice.

Reading input with `input()`

```
first = input("Enter your first name:
")
```

Prompt

The string passed to `input()` tells the user what to enter.

Steps

- 1 display the prompt;
- 2 wait for keyboard input;
- 3 user presses Enter;
- 4 return entered characters as a string.

Reading numeric input

Important

`input()` always returns a string.

Integer input

```
userInput = input("Number of bottles: ")  
bottles = int(userInput)
```

Floating-point input

```
price = float(input("Price: "))
```

Convert input before using it in arithmetic.

Formatting output with %

```
price = 1.215962441314554  
print("%.2f" % price)  
print("%10.2f" % price)
```

Specifier	Meaning
%d	integer
%f	floating-point value
%.2f	2 digits after decimal point
%s	string
%%	literal percent sign

Example: price per ounce

Problem

Compute the price per ounce for a six-pack of cans.

- **Input:** pack price, volume of one can.
- **Process:** compute total volume and price per ounce.
- **Output:** formatted price per ounce.

```
CANS_PER_PACK = 6
packPrice = float(input("Pack price: "))
canVolume = float(input("Can volume: "))
packVolume = canVolume * CANS_PER_PACK
pricePerOunce = packPrice / packVolume
print("Price per ounce: %8.2f" %
      pricePerOunce)
```

Programming Tip 2.4: convert early

Less desirable

```
unitPrice = input("Unit price: ")  
price1 = float(unitPrice)  
price2 = 12 * float(unitPrice)
```

Better

```
unitPrice = float(input("Unit price: "))  
price1 = unitPrice  
price2 = 12 * unitPrice
```

Reason

Early conversion reduces repeated conversion and prevents accidental string arithmetic.

HOW TO 2.1: writing simple programs

Problem: vending machine change

A customer inserts a bill, buys an item, and receives change in dollar coins and quarters. Item prices are multiples of 25 cents.

Inputs

- bill denomination;
- item price in pennies.

Outputs

- number of dollar coins;
- number of quarters.

Vending machine: hand calculation and pseudocode

Concrete example

Bill inserted: \$5.00. Item price: \$2.25.

Change: \$2.75 = 275 pennies.

Return: 2 dollar coins and 3 quarters.

Pseudocode

```
change due = 100 * bill value - item  
price  
dollar coins = change due // 100  
change due = change due % 100  
quarters = change due // 25
```

Vending machine: Python code

```
PENNIES_PER_DOLLAR = 100
PENNIES_PER_QUARTER = 25

billValue = int(input("Enter bill value: "))
itemPrice = int(input("Enter item price in pennies: "))

changeDue = PENNIES_PER_DOLLAR * billValue - itemPrice
dollarCoins = changeDue // PENNIES_PER_DOLLAR
changeDue = changeDue % PENNIES_PER_DOLLAR
quarters = changeDue // PENNIES_PER_QUARTER

print("Dollar coins: %6d" % dollarCoins)
print("Quarters:      %6d" % quarters)
```

Worked Example 2.2: stamps and change

Problem

A stamp machine receives dollar bills and dispenses first-class stamps plus penny stamps as change.

For \$1

$100 // 49 = 2$ first-class stamps.
Change: $100 - 2 * 49 = 2$ cents.

```
FIRST_CLASS_STAMP_PRICE = 49
```

```
dollars = int(input("Dollars: "))  
firstClassStamps = 100 * dollars //  
    FIRST_CLASS_STAMP_PRICE  
change = 100 * dollars -  
    firstClassStamps *  
    FIRST_CLASS_STAMP_PRICE
```

```
print("First class stamps: %6d" %  
    firstClassStamps)  
print("Penny stamps:           %6d" % change  
    )
```

Computing & Society 2.2: bugs in silicon

Historical example

The Pentium FDIV bug showed that some floating-point divisions could produce incorrect results because of a processor hardware defect.

Example idea

A rare division may return a value that is very close to the correct answer, but not exactly correct.

Broader lesson

- software depends on hardware;
- hardware can also contain defects;
- numerical programs need careful testing.

Self Check 2.5: input and output

Concept questions

- 1 What type does `input()` return?
- 2 How can an integer be read directly from input?
- 3 What does `%.2f` mean?

Quick check

Which operators are useful when computing change in whole coin units?

Short programming exercise: simple invoice

Task

Read a product name, unit price, and quantity. Display the total cost.

```
product = input("Product: ")
unitPrice = float(input("Unit price: "))
quantity = int(input("Quantity: "))

total = unitPrice * quantity
print("Product: %s" % product)
print("Total: %.2f" % total)
```

Optional Tool 2.1: SymPy

Main idea

SymPy is a Python package for symbolic mathematics. It can manipulate expressions, not only compute numbers.

```
from sympy import *  
f = sympify("x ** 2 * sin(x)")  
expand((x - 1) * (x + 1))  
solve(x ** 2 + 2 * x - 8)  
diff(f)  
integrate(f)
```

Summary: variables and arithmetic

Variables

- assignment stores a value;
- first assignment initializes a variable;
- = means assignment;
- use descriptive names and constants.

Arithmetic

- /: normal division;
- //: floor division;
- %: remainder;
- **: powers.

Summary: strings and input/output

Strings

- sequence of characters;
- indices start at 0;
- + concatenates, * repeats;
- methods return new strings.

Input/output

- `input()` returns `str`;
- use `int()` and `float()`;
- format output with `%`;
- test with simple values.

Summary questions 1-5

- 1 What does an assignment statement do?
- 2 Which value has type float: 6, 0, 6.0, or -4?
- 3 What is the result of `17 // 5`?
- 4 What is the result of `17 % 5`?
- 5 What does `"Py" + "thon"` produce?

Summary questions 6–10

- 6 Given `s = "Hello"`, what is `s[1]`?
- 7 What type does `input()` return?
- 8 Which statement reads a floating-point value from the user?
- 9 Why should we use named constants?
- 10 What should usually happen before translating a non-trivial computation into Python?

Review exercises

Exercise 1: trace assignments

```
mystery = 1  
mystery = 1 - 2 * mystery  
mystery = mystery + 1
```

Determine the final value of mystery.

Exercise 2: translate math to Python

Write Python expressions for average, powers, and square-root expressions.

Practice exercises

- 1 Convert paper dimensions from inches to millimeters.
- 2 Read a number and display its square, cube, and fourth power.
- 3 Read two integers and print sum, difference, product, average, maximum, and minimum.
- 4 Split a five-digit integer into separate digits.
- 5 Read a word and print first, middle, and last characters.

Key terms

Term	Meaning
Variable	named storage location
Assignment	storing a value in a variable
Data type	representation and valid operations
Expression	combination of values, variables, operators, and calls
String	sequence of characters
Index	position of a character in a string
Prompt	message that tells the user what input is expected
Format specifier	pattern controlling output representation

Integrated short exercise: price per unit

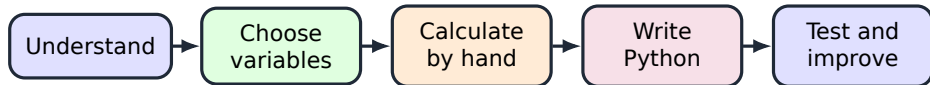
Task

Write a program that reads a product name, package price, and number of units, then displays the price per unit.

```
product = input("Product: ")
packPrice = float(input("Package price: "))
units = int(input("Number of units: "))

pricePerUnit = packPrice / units
print("%s: %.2f per unit" % (product, pricePerUnit))
```

Study guidance



Practice cycle

Predict → calculate by hand → write → run → explain → modify → practice.